

06. 자료 구조와 알고리즘

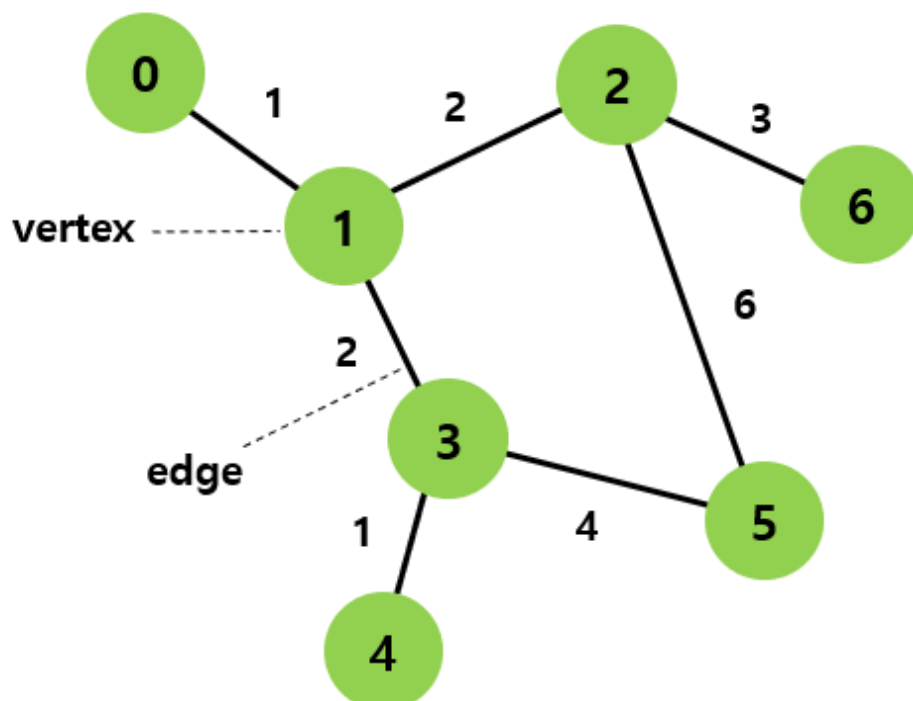
08. 그래프

학습목표

- 그래프가 무엇인지 알 수 있다.
- 그래프를 적재적소에 활용할 수 있다.

들어가며

선형 자료 구조로는 표현할 수 없는 문제가 있다. 회사의 조직도와 같은 계층을 표현한다던가 인스타그램 친구 관계도를 표현하는 것처럼 말이다. 이번 챕터부터는 이런 비선형적 문제를 다룰 수 있는 자료 구조에 대해서 배우게 된다. **그래프(Graph)**는 **자료 간 관계를 표현할 수 있는 자료 구조**로 **정점(Vertex)**과 **간선(Edge)**으로 구성되어 있다. 각 정점은 **고유하게 식별되는 객체(데이터)**이고, 간선은 **정점 간의 관계**를 나타낸다. 이번 시간에는 그래프에 대해서 배워보도록 하자.



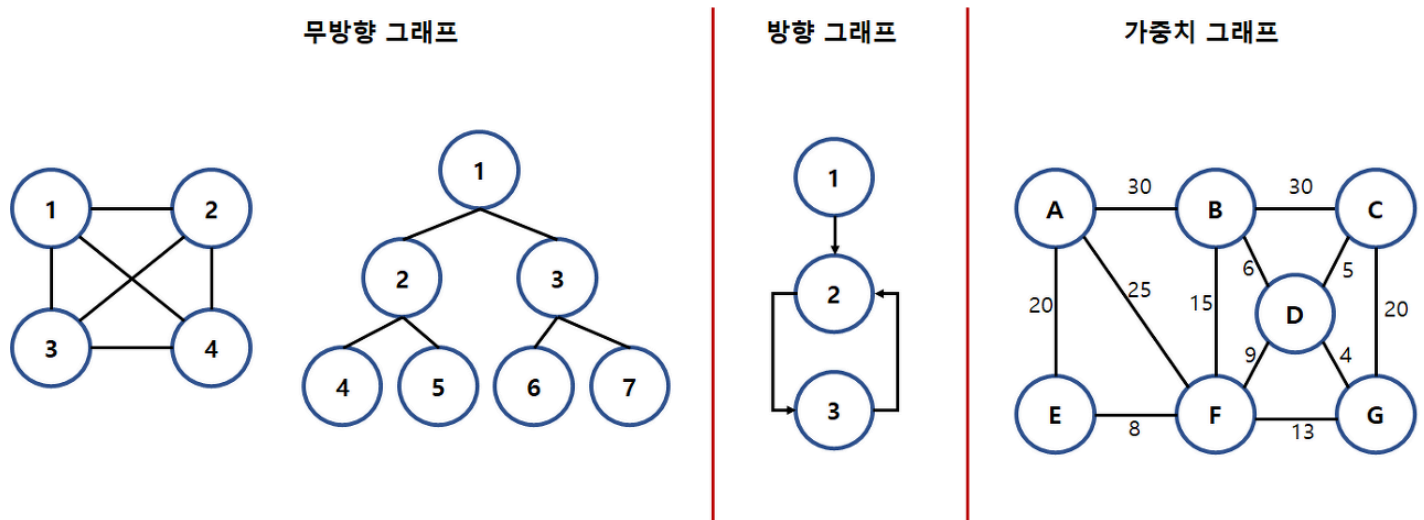
그래프의 모습

종류

그래프는 (1) 간선에 **방향성**이 있는가, (2) 간선에 추가적인 **정보**(가중치)가 있는가에 따라 분류할 수 있다. 간선에 방향성이 있는 것은 **방향 그래프**(Directed Graph)라 하며, 방향성이 없는 것은 **무방향 그래프**(Undirected Graph) 혹은 **양방향 그래프**라고 한다.

간선에는 연결 이상의 정보를 부여할 수 있는데 이를 가중치라 한다. 가중치는 요구 사항에 따라 시간, 비용 등이 될 수 있으며 가중치가 있는 그래프를 **가중치 그래프**(Weighted Graph)*라 한다.

* **네트워크**(Network)라고도 한다.



그래프의 종류

용어

그래프와 관련된 여러 가지 용어가 있다. 먼저 간선으로 연결된 정점끼리는 **인접**(Adjacent)하다고 하며, 해당 간선은 연결된 두 정점에 **부속**(Incident)되어 있다고 한다. **정점에 부속되어 있는 간선의 수**를 **차수**(Degree)라고 하며 정점으로 들어오는 방향인지, 나가는 방향인지 구분해 **진입 차수**(In-degree), **진출 차수**(Out-degree)로 말할 수 있다.

한 정점에서 다른 정점까지 간선으로 연결된 정점을 순서대로 나열한 목록을 **경로**(Path)라 한다. 경로를 구성하는 간선 수를 **경로 길이**(Path length)라 하며, **모두 다른 정점으로 구성된 경로**를 **단순 경로**(Simple Path)라 한다. **단순 경로 중 경로의 시작 정점과 마지막 정점이 같은 경로를 사이클**(Cycle)이라 한다. 또, 경로가 있으면 정점끼리 **연결***(Connected)되었다고 한다.

*모든 정점이 연결되어 있다면 **연결 그래프**(Connected Graph), 아니면 **단절 그래프**(Disconnected Graph)라 한다.

표현

그래프를 표현하는 방법에는 **인접 행렬**(Adjacency Matrix)과 **인접 리스트**(Adjacency List)가 있다. 둘 다 **인접한 정점 사이를 잇는 간선의 정보를 저장**한다. 상기된 그래프를 갖고 인접 행렬과 인접 리스트로 나타내보자.

인접 행렬

인접 행렬은 정점이 N개 있을 경우 N x N 크기의 **2차원 배열**로 그래프를 표현한 것이며, 배열의 원소는 간선의 **가중치**를 저장한다. 연결되어 있지 않은 경우 **0** 혹은 **무한대 값**으로 표현한다. **연결되지 않은 정보도 저장하고 있기 때문에 그래프에 간선이 적다면* 그만큼 메모리를 낭비하게 된다.**

*이를 **희소 그래프**(Sparse Graph)라 한다. 반대는 **밀집 그래프**(Dense Graph)다.

```
// 정점이 7개이므로 7 x 7의 2차원 배열을 정의한다.
int[,] graph = new int[7, 7];

// 무방향 그래프이므로 서로 연결해준다.
graph[0, 1] = 1; // 0 -> 1의 방향과 1의 가중치를 가진 간선
graph[1, 0] = 1; // 1 -> 0의 방향과 1의 가중치를 가진 간선

graph[1, 2] = 2; // 1 -> 2의 방향과 2의 가중치를 가진 간선
graph[2, 1] = 2; // 2 -> 1의 방향과 2의 가중치를 가진 간선
graph[1, 3] = 2; // 1 -> 3의 방향과 2의 가중치를 가진 간선
graph[3, 1] = 2; // 3 -> 1의 방향과 2의 가중치를 가진 간선

// 이하 설명 생략
graph[2, 5] = 6;
graph[5, 2] = 6;
graph[2, 6] = 3;
graph[6, 2] = 3;

graph[3, 4] = 1;
graph[4, 3] = 1;
graph[3, 5] = 4;
graph[5, 3] = 4;
```

인접 리스트

인접 리스트는 **연결된 간선만을 저장**한다.

```
// 진출 간선 타입을 정의한다.
```

```

struct OutEdge
{
    public int Vertex;
    public int Weight;
}

// 정점이 7개이므로 크기가 7인 리스트를 만든다.
List<OutEdge>[] graph = new List<OutEdge>[7];

graph[0].Add(new OutEdge(){ Vertex = 1, Weight = 1 });

graph[1].Add(new OutEdge(){ Vertex = 0, Weight = 1 });
graph[1].Add(new OutEdge(){ Vertex = 2, Weight = 2 });
graph[1].Add(new OutEdge(){ Vertex = 3, Weight = 2 });

graph[2].Add(new OutEdge(){ Vertex = 1, Weight = 2 });
graph[2].Add(new OutEdge(){ Vertex = 5, Weight = 6 });
graph[2].Add(new OutEdge(){ Vertex = 6, Weight = 3 });

graph[3].Add(new OutEdge(){ Vertex = 1, Weight = 2 });
graph[3].Add(new OutEdge(){ Vertex = 4, Weight = 1 });
graph[3].Add(new OutEdge(){ Vertex = 5, Weight = 4 });

graph[4].Add(new OutEdge(){ Vertex = 3, Weight = 1 });

graph[5].Add(new OutEdge(){ Vertex = 2, Weight = 6 });
graph[5].Add(new OutEdge(){ Vertex = 3, Weight = 4 });

graph[6].Add(new OutEdge(){ Vertex = 2, Weight = 3 });

```

더해보기

1. 그래프와 스택/큐와 같은 선형 자료구조의 차이점을 설명하고, 그래프가 필요한 구체적인 사례를 하나 드시오.
2. 그래프의 종류는 어떻게 되나요?
3. 그래프와 관련된 용어에 대해서 정리해보세요.
4. 가중치 그래프(네트워크)에서 '가중치'가 실생활 문제(예: 내비게이션)에서 어떤 의미로 활용될 수 있는지 설명하시오.
5. 정점의 개수가 N 개일 때, 두 정점의 연결 여부를 $O(1)$ 의 시간 복잡도로 즉시 확인할 수 있는 저장 방식은 무엇인가?

6. '인접 행렬' 방식이 '인접 리스트' 방식에 비해 메모리 효율 면에서 불리한 이유는 무엇인지 정점이 많고 간선이 적은 상황(희소 그래프)을 예로 들어 설명하시오.

참고자료

- [C로 배우는 쉬운 자료구조](#)
- [누구나 자료 구조와 알고리즘](#)